

ABSTRACT

Cloud computing has become an essential technology for hosting scalable, reliable, and fault-tolerant applications. Modern organizations require highly available web applications that remain operational even when server failures occur or user traffic increases significantly.

This project demonstrates the deployment of a Highly Available Web Application using Amazon Web Services (AWS). The implementation utilizes Amazon EC2 instances for web hosting, Amazon Machine Images (AMI) for reusable server configurations, Launch Templates for standardized instance creation, Classic Load Balancer (CLB) for distributing incoming traffic, and Auto Scaling Groups (ASG) to automatically maintain the desired number of running instances.

Apache HTTP Server is installed on Amazon Linux to host a Bootstrap-based website. CloudWatch continuously monitors CPU utilization, while Amazon SNS sends real-time email notifications whenever predefined threshold values are crossed. High availability is validated by manually terminating an EC2 instance and observing the automatic creation of a replacement instance without affecting application availability.

The project successfully demonstrates cloud infrastructure automation, load balancing, monitoring, failover recovery, and fault tolerance using AWS services. It provides practical exposure to designing reliable cloud-native applications while improving infrastructure availability, scalability, and operational efficiency.

Keywords: AWS, Amazon EC2, Auto Scaling Group, Classic Load Balancer, AMI, Launch Template, CloudWatch, SNS, Apache Web Server, High Availability, Cloud Computing.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	4
ABSTRACT	5
LIST OF FIGURES	6
LIST OF ABBREVIATIONS.....	7
 1. INTRODUCTION.....	 11
1.1 Introduction.....	11
 2. LITERATURE SURVEY	 12
2.1 Introduction.....	12
2.2 Literature Review	12
2.3 Summary	14
 3. SYSTEM ANALYSIS	 15
3.1 Introduction.....	15
3.2 Existing System.....	15
3.3 Limitations of Existing System.....	16
3.4 Proposed System	16
3.5 Advantages of Proposed System	17
3.6 Functional Requirements	18
3.7 Non-Functional Requirements	18
3.8 Feasibility Study.....	19
3.9 System Flow	19
3.10 Summary	19
 4. SYSTEM DESIGN	 20
4.1 Introduction.....	20
4.2 Overall System Architecture.....	20
4.3 Design Modules.....	21
4.4 Workflow of Proposed System	24
4.5 Security Design	25
4.6 Fault Tolerance Design.....	26

4.7 Scalability Design.....	26
4.8 High Availability Design	26
4.9 Summary	26
5. SYSTEM REQUIREMENTS AND AWS SERVICES USED.....	27
5.1 Introduction.....	27
5.2 Hardware Requirements	27
5.3 Software Requirements.....	27
5.4 AWS Services Used.....	28
5.5 Tools and Technologies Used	31
5.6 Project Workflow	31
5.7 Advantages of AWS Services Used	32
5.8 Summary	32
6. IMPLEMENTATION	33
6.1 Introduction.....	33
6.2 AWS Management Console	33
6.3 Creating an EC2 Instance.....	33
6.4 Connecting to EC2	34
6.5 Installing Apache Web Server.....	35
6.6 Deploying the Website.....	35
6.7 Creating an Amazon Machine Image (AMI)	36
6.8 Creating Launch Template.....	37
6.9 Creating Classic Load Balancer	37
6.10 Creating Auto Scaling Group.....	38
6.11 Verifying Website through the Load Balancer	39
6.12 Configuring CloudWatch Alarm	40
6.13 Creating SNS Topic	41
6.14 High Availability Testing.....	41
6.15 Monitoring Results	42
6.16 Implementation Summary.....	43
7. TESTING AND RESULTS.....	43
7.1 Introduction.....	43
7.2 Testing Objectives	43

7.3 Testing Environment	44
7.4 Functional Testing	44
7.5 High Availability Testing	46
7.6 CloudWatch Monitoring Test	47
7.7 SNS Notification Test.....	47
7.8 Load Balancer Verification.....	47
7.9 Performance Analysis.....	48
7.10 Result Analysis.....	48
7.11 Summary	48
8. CONCLUSION AND FUTURE ENHANCEMENT.....	49
8.1 Conclusion	49
8.2 Future Enhancement	50
8.3 Overall Project Outcome.....	52
REFERENCES.....	53
APPENDIX.....	54

LIST OF FIGURES

- 3.1 Existing System Architecture
- 3.2 Proposed System Architecture / System Flow
- 4.1 Overall System Architecture Diagram
- 6.1 AWS Management Console Home Page
- 6.2 EC2 Instance Creation
- 6.3 SSH Connection and Apache Installation
- 6.4 Hosted Web Application
- 6.5 Amazon Machine Image Creation
- 6.6 Launch Template Configuration
- 6.7 Classic Load Balancer Configuration
- 6.8 Auto Scaling Group Configuration
- 6.9 Website Accessed through Load Balancer DNS
- 6.10 CloudWatch Alarm Configuration
- 7.1 CloudWatch Monitoring Graph
- 7.2 Alarm Notification Email

LIST OF ABBREVIATIONS

Abbreviation	Description
AWS	Amazon Web Services
EC2	Elastic Compute Cloud
AMI	Amazon Machine Image
ASG	Auto Scaling Group
CLB	Classic Load Balancer
VPC	Virtual Private Cloud
IAM	Identity and Access Management
SNS	Simple Notification Service
SG	Security Group
CPU	Central Processing Unit
HTTP	Hypertext Transfer Protocol
DNS	Domain Name System
SSH	Secure Shell

CHAPTER 1

INTRODUCTION

1.1 Introduction

Cloud computing has transformed the way organizations deploy, manage, and maintain web applications by providing scalable, reliable, and cost-effective infrastructure over the Internet. Instead of investing in expensive physical servers, organizations can leverage cloud platforms such as Amazon Web Services (AWS) to provision computing resources on demand. AWS offers a comprehensive set of cloud services that simplify application deployment while ensuring high availability, fault tolerance, security, and scalability.

A highly available web application is designed to remain operational even when one or more infrastructure components fail. This is achieved by distributing incoming user traffic across multiple servers using load balancing techniques and automatically replacing failed instances through auto scaling mechanisms. High availability minimizes downtime, improves user experience, and ensures uninterrupted service delivery.

In this project, a highly available web application is deployed using Amazon EC2 instances running the Apache HTTP Server. The web application is hosted using a Bootstrap-based website template. An Amazon Machine Image (AMI) is created to capture the configured server environment, and a Launch Template is used to standardize future instance launches. A Classic Load Balancer (CLB) distributes incoming HTTP requests across multiple EC2 instances, while an Auto Scaling Group (ASG) automatically launches additional instances when required and replaces unhealthy instances in case of failures.

Amazon CloudWatch continuously monitors CPU utilization and generates alarms when predefined thresholds are exceeded. Amazon Simple Notification Service (SNS) sends email notifications to administrators whenever alarm conditions occur, enabling proactive infrastructure monitoring. The complete deployment demonstrates how AWS services work together to provide a reliable, scalable, and fault-tolerant cloud infrastructure.

This project provides practical knowledge of cloud computing concepts, infrastructure automation, monitoring, and high availability while exposing learners to real-world AWS deployment practices used by modern enterprises.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Cloud computing has significantly transformed the deployment and management of web applications by providing scalable, reliable, and cost-effective infrastructure. Traditional hosting methods depend on single physical servers, which often suffer from limited scalability, hardware failures, and increased maintenance costs. With the rapid growth of cloud platforms such as Amazon Web Services (AWS), organizations can deploy highly available applications capable of handling increasing user traffic while minimizing downtime.

High availability has become one of the primary objectives of modern cloud infrastructure. AWS provides several services, including Amazon EC2, Auto Scaling Groups, Elastic Load Balancing, Amazon Machine Images (AMI), CloudWatch, and Simple Notification Service (SNS), that work together to build resilient cloud environments. Numerous researchers and organizations have proposed various techniques for improving application availability, fault tolerance, monitoring, and automated recovery.

This literature survey reviews existing research related to cloud computing, load balancing, auto scaling, infrastructure monitoring, and high availability. The analysis highlights the advantages and limitations of current approaches and demonstrates how the proposed AWS-based implementation addresses these challenges through automated cloud infrastructure management.

2.2 Literature Review

2.2.1 Amazon Web Services – Auto Scaling and Elastic Load Balancing

Amazon Web Services introduced Auto Scaling and Elastic Load Balancing to improve application availability and efficiently manage changing workloads. Auto Scaling automatically launches or terminates EC2 instances based on predefined policies, while Load Balancers distribute incoming client requests among multiple servers.

The combination of these services improves fault tolerance, reduces downtime, and optimizes infrastructure costs by provisioning resources only when required. However, proper configuration of scaling policies and monitoring thresholds is necessary to achieve optimal performance.

2.2.2 VMware – Cloud Infrastructure Management (2022)

VMware proposed cloud infrastructure management techniques that emphasize virtualization, workload balancing, and automated resource allocation. Their approach focuses on improving application reliability by distributing workloads across multiple virtual machines while maintaining system availability.

Although virtualization improves resource utilization, infrastructure management becomes more complex when deployed across multiple cloud environments. Organizations often require additional monitoring and automation tools to manage dynamic workloads effectively.

2.2.3 Microsoft Azure – High Availability Architecture (2023)

Microsoft Azure provides several cloud-native services for building highly available applications, including Virtual Machine Scale Sets, Azure Load Balancer, and Azure Monitor. These services automatically distribute user traffic and monitor infrastructure health to reduce service interruptions.

Azure's high availability model demonstrates significant improvements in reliability and scalability. However, organizations using multi-cloud strategies may require additional configuration and interoperability between different cloud providers.

2.2.4 Google Cloud Platform – Cloud Monitoring and Scaling (2024)

Google Cloud Platform integrates Compute Engine, Managed Instance Groups, Cloud Load Balancing, and Cloud Monitoring to support scalable web applications. The platform continuously analyzes resource utilization and automatically adjusts computing resources according to workload demand.

This approach improves operational efficiency and resource optimization. Nevertheless, configuring monitoring metrics and scaling policies requires careful planning to prevent unnecessary resource allocation.

2.2.5 IBM Cloud – Infrastructure Automation (2024)

IBM Cloud introduced infrastructure automation using reusable machine images, deployment templates, and automated monitoring services. Infrastructure automation minimizes manual server configuration and accelerates application deployment while maintaining consistency across multiple environments.

Although automation reduces administrative effort, organizations must implement strong security policies and monitoring mechanisms to protect cloud resources from unauthorized access and configuration errors.

Table 2.1 Comparative Literature Review

Year	Author / Organization	Technology	Problem Addressed	Conclusion
2021	Amazon Web Services	Auto Scaling & Load Balancer	Dynamic workload management	Improved scalability and fault tolerance
2022	VMware	Virtual Infrastructure	Resource optimization	Better virtualization with increased management complexity

Year	Author / Organization	Technology	Problem Addressed	Conclusion
2023	Microsoft Azure	Virtual Machine Scale Sets	High availability	Reliable infrastructure with automatic scaling
2024	Google Cloud	Managed Instance Groups	Infrastructure monitoring	Efficient resource utilization through automation
2024	IBM Cloud	Infrastructure Automation	Deployment consistency	Reduced manual effort with automated provisioning

2.3 Summary

The literature survey demonstrates that cloud computing platforms have made significant progress in delivering scalable, highly available, and fault-tolerant infrastructure. Existing solutions emphasize load balancing, automated scaling, infrastructure monitoring, and resource optimization to improve application performance and minimize downtime.

However, many implementations focus only on individual cloud services rather than demonstrating a complete end-to-end deployment process. The proposed project integrates Amazon EC2, Amazon Machine Images (AMI), Launch Templates, Classic Load Balancer, Auto Scaling Group, CloudWatch, and Amazon SNS into a unified cloud architecture. This integrated approach enables automated application deployment, real-time monitoring, automatic recovery from server failures, and continuous service availability.

The proposed implementation provides practical exposure to enterprise cloud infrastructure by combining automation, monitoring, and fault tolerance into a single deployment environment, thereby meeting modern cloud computing requirements.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction

System analysis is one of the most important phases of software development because it helps identify the limitations of the existing system and provides a structured approach for designing an improved solution. In cloud computing, system analysis involves evaluating the current infrastructure, identifying performance bottlenecks, understanding scalability requirements, and selecting appropriate cloud services to improve application availability and reliability.

The proposed project focuses on deploying a highly available web application using Amazon Web Services (AWS). The system is designed to eliminate the drawbacks of traditional single-server hosting by implementing automatic load balancing, health monitoring, fault tolerance, and infrastructure automation.

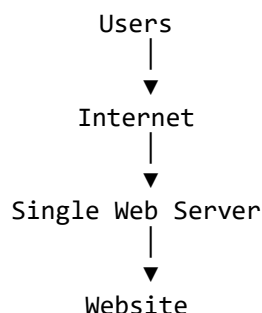
3.2 Existing System

In traditional web hosting environments, websites are generally deployed on a single physical or virtual server. All incoming user requests are processed by that server alone. As the number of users increases, the server experiences higher CPU utilization, memory consumption, and network traffic, eventually reducing application performance.

If the server fails due to hardware malfunction, software crashes, operating system failure, or network issues, the entire application becomes unavailable until manual intervention is performed. Administrators must manually replace failed servers, restore backups, and reconfigure services, resulting in increased downtime and operational costs.

Furthermore, traditional systems lack automatic monitoring, resource scaling, and centralized infrastructure management, making them unsuitable for modern applications that require continuous availability.

Existing System Architecture



[Figure 3.1: Existing System Architecture]

3.3 Limitations of Existing System

The traditional hosting approach suffers from several disadvantages:

- Single Point of Failure
- No Automatic Recovery
- Limited Scalability
- High Downtime
- Manual Server Maintenance
- Poor Resource Utilization
- No Automatic Monitoring
- Increased Operational Cost
- Difficult Infrastructure Expansion
- Reduced Application Availability

These limitations motivate organizations to migrate their applications to cloud platforms capable of providing automated infrastructure management.

3.4 Proposed System

The proposed system deploys the web application on Amazon Web Services using multiple cloud services working together to provide high availability.

Initially, an Amazon EC2 instance is launched inside a Virtual Private Cloud (VPC). Apache HTTP Server is installed, and the web application is deployed. After verifying successful deployment, an Amazon Machine Image (AMI) is created from the configured instance.

A Launch Template is then created using the AMI. The Launch Template defines the instance type, security group, key pair, IAM role, and boot configuration.

An Auto Scaling Group is configured using the Launch Template. The Auto Scaling Group automatically maintains the desired number of running instances and replaces unhealthy instances whenever failures occur.

A Classic Load Balancer distributes incoming client requests across multiple EC2 instances. This ensures that no single server becomes overloaded.

Amazon CloudWatch continuously monitors infrastructure metrics such as CPU utilization. When predefined thresholds are exceeded, CloudWatch generates alarms.

Amazon SNS delivers email notifications whenever CloudWatch alarms are triggered, enabling administrators to respond quickly to infrastructure issues.

The combination of these AWS services provides continuous application availability, automatic scaling, and fault tolerance.

3.5 Advantages of Proposed System

The proposed AWS-based infrastructure provides several advantages over traditional hosting.

High Availability

The application remains available even if one EC2 instance fails because Auto Scaling automatically launches a replacement instance.

Automatic Load Distribution

Classic Load Balancer distributes incoming requests among healthy EC2 instances, improving response time.

Fault Tolerance

Failed instances are automatically detected and replaced without manual intervention.

Automatic Scaling

Infrastructure automatically scales according to workload, reducing performance bottlenecks.

Continuous Monitoring

CloudWatch continuously monitors resource utilization and system health.

Real-Time Alerts

SNS immediately sends email notifications whenever CloudWatch alarms are triggered.

Infrastructure Automation

Launch Templates and AMIs eliminate repetitive server configuration tasks.

Cost Optimization

Resources are automatically adjusted according to application demand, minimizing unnecessary infrastructure costs.

Improved Security

AWS Security Groups and IAM Roles provide secure access control and resource protection.

Better User Experience

Users experience uninterrupted access to the web application even during server failures or increased traffic.

3.6 Functional Requirements

The proposed system must satisfy the following functional requirements:

- Deploy web application on Amazon EC2
- Host website using Apache Web Server
- Create Amazon Machine Image (AMI)
- Configure Launch Template
- Configure Auto Scaling Group
- Configure Classic Load Balancer
- Configure CloudWatch Monitoring
- Configure SNS Email Notification
- Replace unhealthy EC2 instances automatically
- Monitor CPU Utilization
- Distribute client requests across multiple instances
- Maintain continuous application availability

3.7 Non-Functional Requirements

The non-functional requirements include:

Reliability

The application should remain operational even during server failures.

Scalability

Infrastructure should automatically increase or decrease resources depending on workload.

Availability

Application uptime should be maximized.

Performance

Load balancing should improve response time.

Security

Only authorized users should access AWS resources.

Maintainability

Infrastructure should be easy to manage using reusable templates.

Portability

The deployment should be reproducible across AWS Regions.

3.8 Feasibility Study

Technical Feasibility

AWS provides all required cloud services, including EC2, Auto Scaling, Load Balancer, CloudWatch, IAM, and SNS. Therefore, the proposed project is technically feasible.

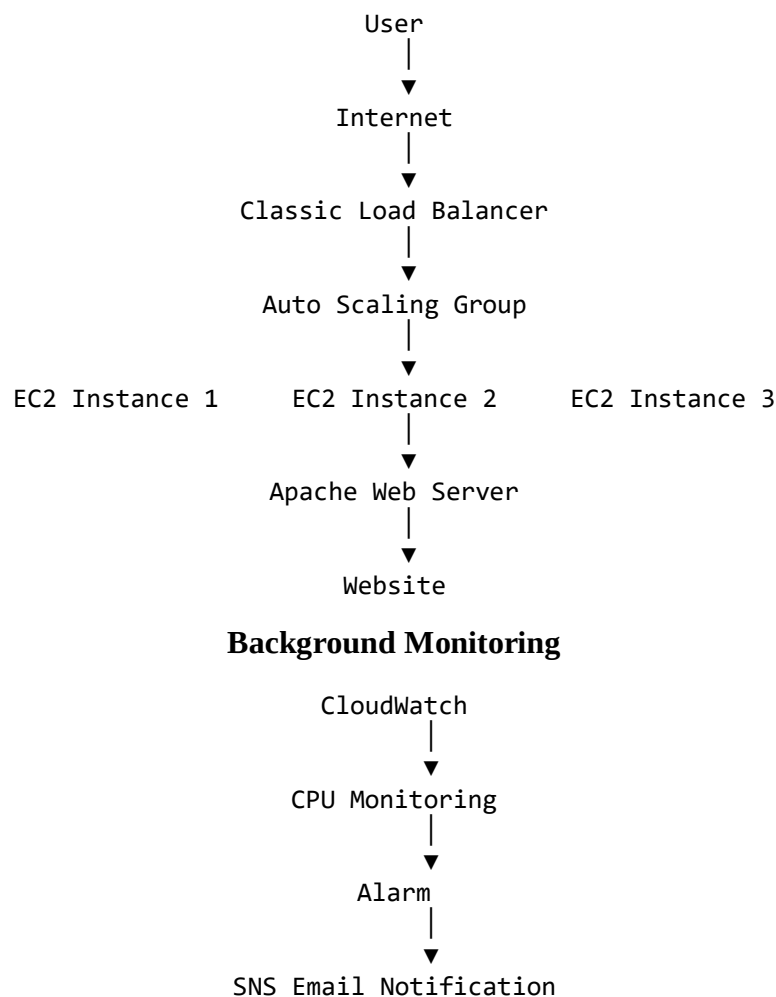
Economic Feasibility

AWS Free Tier allows students and developers to deploy cloud applications with minimal cost. Infrastructure automation further reduces operational expenses by utilizing resources only when required.

Operational Feasibility

The proposed system requires minimal manual intervention because monitoring, scaling, and recovery are fully automated.

3.9 System Flow



[Figure 3.2: Proposed System Flow]

3.10 Summary

The system analysis demonstrates that traditional single-server hosting environments cannot satisfy the availability, scalability, and fault tolerance requirements of modern cloud applications. The proposed AWS-based solution overcomes these limitations by integrating Amazon EC2, AMI, Launch Templates, Auto Scaling Groups, Classic Load Balancer, CloudWatch, and SNS into a unified cloud infrastructure.

The system automatically distributes incoming traffic, replaces failed instances, monitors infrastructure health, and sends real-time notifications, thereby ensuring continuous application availability and improved operational efficiency.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

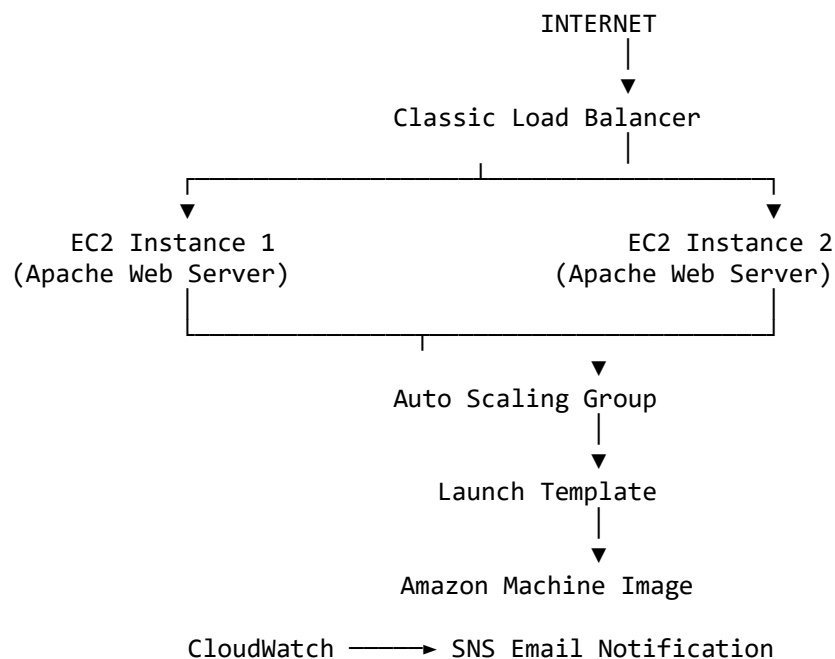
System design is the process of defining the architecture, modules, interfaces, and workflow of a software system to satisfy the functional and non-functional requirements identified during system analysis. It provides a blueprint for implementing the proposed solution by specifying how different system components interact with one another.

In this project, the system is designed to deploy a Highly Available Web Application on Amazon Web Services (AWS). The architecture integrates Amazon EC2, Amazon Machine Images (AMI), Launch Templates, Auto Scaling Groups (ASG), Classic Load Balancer (CLB), Amazon CloudWatch, Amazon Simple Notification Service (SNS), IAM, Security Groups, and Virtual Private Cloud (VPC). These services work together to provide scalability, fault tolerance, automated monitoring, and continuous application availability.

The design ensures that incoming client requests are distributed across multiple web servers while monitoring system health and automatically recovering from failures.

4.2 Overall System Architecture

The proposed architecture follows a multi-tier cloud deployment model where user requests are distributed through a load balancer before reaching multiple EC2 instances managed by an Auto Scaling Group.



[Figure 4.1: Overall System Architecture]

Description

The architecture consists of the following components:

- Internet users access the website through HTTP.
- Classic Load Balancer distributes incoming traffic.
- Multiple EC2 instances process requests.
- Auto Scaling maintains the desired number of instances.
- Launch Template standardizes instance creation.
- AMI stores the configured server image.
- CloudWatch monitors CPU utilization.
- SNS sends email alerts when alarms are triggered.

4.3 Design Modules

The proposed system consists of eight major modules.

Module 1 – Amazon EC2 Instance

Amazon Elastic Compute Cloud (EC2) provides virtual servers for hosting the web application. The instance runs Amazon Linux and Apache HTTP Server.

Responsibilities

- Launch virtual machine
- Install Apache Web Server
- Host Bootstrap website
- Configure Security Groups
- Connect using SSH

Module 2 – Apache Web Server

Apache HTTP Server hosts the website and responds to HTTP requests from users.

Responsibilities

- Serve HTML pages
- Process HTTP requests
- Deliver website content
- Handle client connections

Module 3 – Amazon Machine Image (AMI)

An Amazon Machine Image captures the complete configuration of the EC2 instance.

Responsibilities

- Store server configuration
- Preserve installed software
- Create reusable server images
- Enable automatic instance creation

Module 4 – Launch Template

Launch Templates define how new EC2 instances should be launched.

Configuration Includes

- Instance Type
- AMI ID
- Key Pair
- Security Group
- IAM Role
- Storage Configuration

Launch Templates eliminate repetitive configuration during Auto Scaling.

Module 5 – Auto Scaling Group

Auto Scaling maintains application availability by automatically launching or terminating EC2 instances.

Functions

- Replace failed instances
- Scale based on demand
- Maintain desired capacity
- Improve application availability

Example Configuration:

Parameter	Value
Minimum Instances	2
Desired Instances	2
Maximum Instances	4

Module 6 – Classic Load Balancer

Classic Load Balancer distributes incoming traffic across multiple healthy EC2 instances.

Responsibilities

- Health Check

- Request Distribution
- Fault Tolerance
- Session Handling

Benefits

- Improved Response Time
- Reduced Server Overload
- Better Resource Utilization

Module 7 – CloudWatch Monitoring

Amazon CloudWatch continuously collects system metrics.

Metrics Monitored

- CPU Utilization
- Network Traffic
- Disk Activity
- Instance Status

CloudWatch automatically evaluates metrics against predefined thresholds.

Module 8 – SNS Notification

Amazon Simple Notification Service (SNS) provides real-time alert notifications.

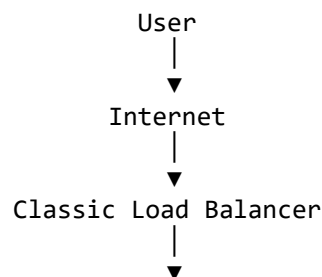
Responsibilities

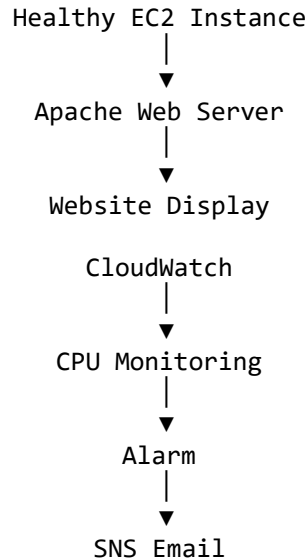
- Send Email Alerts
- Notify Administrators
- Trigger Alarm Messages
- Report Infrastructure Issues

Whenever CPU utilization exceeds the configured threshold, CloudWatch activates SNS to deliver an email notification.

4.4 Workflow of Proposed System

The overall execution flow is shown below.





Workflow Description

- User accesses the website through a web browser.
- Request reaches the Classic Load Balancer.
- Load Balancer forwards the request to a healthy EC2 instance.
- Apache processes the request.
- Website is displayed to the user.
- CloudWatch continuously monitors CPU utilization.
- If CPU exceeds the threshold, CloudWatch triggers an alarm.
- SNS sends an email notification.
- If an EC2 instance fails, Auto Scaling launches a replacement automatically.

4.5 Security Design

Security is one of the most critical aspects of cloud infrastructure. The following AWS security mechanisms are implemented.

IAM

Identity and Access Management controls access to AWS resources through users, groups, roles, and policies.

Example Policies

- AmazonEC2FullAccess
- CloudWatchFullAccess
- AutoScalingFullAccess
- AmazonSNSFullAccess

Security Groups

Security Groups act as virtual firewalls.

Protocol	Port	Purpose
SSH	22	Remote Login
HTTP	80	Website Access
HTTPS	443	Secure Access

Key Pair Authentication

SSH access is protected using an AWS-generated Key Pair.

Advantages

- Secure authentication
- No password exposure
- Encrypted connection

4.6 Fault Tolerance Design

Fault tolerance ensures uninterrupted application availability.

The proposed system achieves fault tolerance using:

- Auto Scaling
- Health Checks
- Load Balancer
- AMI
- Launch Template

If an EC2 instance becomes unhealthy:

- Load Balancer stops routing requests.
- Auto Scaling terminates the unhealthy instance.
- Auto Scaling launches a new instance.
- New instance joins the Load Balancer.
- Users experience minimal or no downtime.

4.7 Scalability Design

The application automatically adjusts computing resources based on demand.

Scaling Out

- Launch additional EC2 instances
- Handle more user requests

Scaling In

- Terminate unused instances
- Reduce infrastructure cost

This elastic behavior ensures efficient resource utilization.

4.8 High Availability Design

High Availability is achieved using multiple AWS services.

Features include:

- Multiple EC2 Instances
- Health Checks
- Automatic Recovery
- Load Balancing
- Continuous Monitoring
- Automatic Scaling

Even if one server becomes unavailable, another server continues serving user requests without interruption.

4.9 Summary

The system design chapter describes the complete architecture of the proposed cloud infrastructure. The design integrates EC2, AMI, Launch Templates, Auto Scaling Groups, Classic Load Balancer, CloudWatch, SNS, IAM, Security Groups, and Apache Web Server into a unified architecture capable of providing scalable, secure, and highly available web application hosting.

The modular design simplifies infrastructure management while ensuring automatic recovery, efficient resource utilization, and continuous service availability.

CHAPTER 5

SYSTEM REQUIREMENTS AND AWS SERVICES USED

5.1 Introduction

A successful cloud deployment depends on selecting appropriate hardware, software, networking components, and cloud services. Amazon Web Services (AWS) provides a flexible environment where computing resources can be provisioned on demand without requiring organizations to purchase or maintain physical servers.

This project deploys a highly available web application using AWS cloud services. The implementation uses Amazon EC2 for hosting the application, Apache HTTP Server for serving web pages, Auto Scaling Group for automatic instance management, Classic Load Balancer for traffic distribution, CloudWatch for monitoring, and SNS for real-time notifications. Together, these services provide a scalable, reliable, and fault-tolerant cloud infrastructure.

5.2 Hardware Requirements

Although AWS provides virtual infrastructure, the development machine used to access AWS Management Console and configure resources should meet the following minimum requirements.

Component	Specification
Processor	Intel Core i3/i5/i7 or AMD Equivalent
RAM	Minimum 8 GB
Storage	100 GB Free Disk Space
Internet Connection	Broadband Connection
Monitor	1366 × 768 Resolution or Higher
Keyboard & Mouse	Standard Input Devices

5.3 Software Requirements

The following software components were used during project implementation.

Software	Purpose
Windows 10 / Windows 11	Development Environment
Amazon Linux 2	EC2 Operating System
Apache HTTP Server	Website Hosting
AWS Management Console	Cloud Resource Management
PuTTY / MobaXterm / Windows Terminal	SSH Access
Git (Optional)	Version Control

Software	Purpose
Visual Studio Code	HTML Editing
Google Chrome / Microsoft Edge	Web Browser

5.4 AWS Services Used

The project utilizes several AWS services that work together to build a highly available cloud infrastructure.

5.4.1 Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 provides virtual servers in the cloud. It allows users to launch Linux or Windows virtual machines with customizable CPU, memory, storage, and networking resources.

Purpose in this Project

- Host Apache Web Server
- Deploy Bootstrap Website
- Execute Linux Commands
- Handle Client Requests

Advantages

- On-demand provisioning
- Highly scalable
- Flexible instance selection
- Secure networking
- Easy integration with AWS services

5.4.2 Amazon Machine Image (AMI)

An AMI is a preconfigured image containing the operating system, installed software, and application configuration.

Purpose

- Backup configured server
- Launch identical EC2 instances
- Reduce deployment time
- Ensure configuration consistency

AMI Created: WebServer-AMI

5.4.3 Launch Template

Launch Templates simplify EC2 deployment by storing instance configuration.

The Launch Template includes:

- AMI ID
- Instance Type
- Key Pair
- Security Group
- IAM Role
- Storage Settings

Launch Template Name: WebTemplate

Advantages

- Faster deployment
- Consistent configuration
- Simplifies Auto Scaling

5.4.4 Auto Scaling Group (ASG)

Auto Scaling automatically manages the number of EC2 instances.

Parameter	Value
Minimum Capacity	2
Desired Capacity	2
Maximum Capacity	4

Benefits

- Automatic recovery
- High availability
- Automatic scaling
- Cost optimization

Auto Scaling Group Name: Web-ASG

5.4.5 Classic Load Balancer (CLB)

Classic Load Balancer distributes incoming HTTP traffic among multiple EC2 instances.

Functions

- Health Check
- Traffic Distribution
- Failover Support
- Load Balancing

Load Balancer Name: WebCLB

5.4.6 Amazon CloudWatch

CloudWatch continuously monitors AWS resources and collects performance metrics.

Metrics

- CPU Utilization
- Network In
- Network Out
- Disk Read
- Disk Write
- Instance Status

Alarm Created: webalarm

CloudWatch automatically triggers alarms whenever CPU utilization exceeds the configured threshold.

5.4.7 Amazon SNS (Simple Notification Service)

SNS provides notification services using Email, SMS, and HTTP endpoints.

Purpose

- Email Alerts
- Alarm Notification
- Infrastructure Monitoring

SNS Topic Name: WebAlert

Whenever CloudWatch detects abnormal CPU usage, SNS immediately sends an email notification to the administrator.

5.4.8 IAM (Identity and Access Management)

IAM controls authentication and authorization for AWS resources.

IAM Features

- Users
- Groups
- Roles
- Policies
- Multi-Factor Authentication (MFA)

Policies Used

- AmazonEC2FullAccess
- AutoScalingFullAccess
- CloudWatchFullAccess
- AmazonSNSFullAccess

5.4.9 Virtual Private Cloud (VPC)

Amazon VPC provides an isolated virtual network where AWS resources are deployed securely.

Components

- Public Subnet
- Internet Gateway
- Route Table
- Security Groups
- Network ACL

The EC2 instances used in this project are deployed within a VPC to ensure secure communication and controlled network access.

5.4.10 Security Groups

Security Groups act as virtual firewalls.

Protocol	Port	Purpose
SSH	22	Remote Access
HTTP	80	Website Access
HTTPS	443	Secure Website

5.5 Tools and Technologies Used

Tool	Purpose
HTML5	Website Structure
CSS3	Styling
Bootstrap	Responsive Design
Apache HTTP Server	Web Hosting
Amazon Linux	Operating System
AWS CLI (Optional)	AWS Management
SSH	Remote Administration

5.6 Project Workflow

The implementation follows the sequence below:

- Create AWS Account.
- Launch EC2 Instance.
- Install Apache HTTP Server.
- Upload Website Files.
- Verify Website.
- Create Amazon Machine Image.
- Create Launch Template.
- Configure Classic Load Balancer.
- Configure Auto Scaling Group.
- Configure CloudWatch Alarm.
- Configure SNS Topic.
- Verify Email Subscription.
- Test Auto Scaling.
- Terminate EC2 Instance.
- Observe Automatic Recovery.

5.7 Advantages of AWS Services Used

AWS Service	Major Advantage
EC2	Virtual Server Hosting
AMI	Quick Server Deployment
Launch Template	Reusable Configuration
Auto Scaling	Automatic Recovery
Classic Load Balancer	Traffic Distribution
CloudWatch	Performance Monitoring
SNS	Instant Notifications
IAM	Secure Access Control
VPC	Network Isolation
Security Group	Firewall Protection

5.8 Summary

This chapter presented the hardware and software requirements, AWS cloud services, and development tools used in the project. Each AWS service plays a specific role in ensuring the successful deployment of a highly available web application. Amazon EC2 hosts the application, AMI and Launch Templates simplify deployment, Auto Scaling maintains application availability, the Classic Load Balancer distributes traffic, CloudWatch monitors infrastructure health, SNS

provides real-time alerts, IAM secures access, and VPC with Security Groups ensures network protection.

Together, these services create a reliable, scalable, secure, and fault-tolerant cloud environment suitable for modern web applications.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

Implementation is the phase where the proposed system is developed and deployed using the selected technologies. In this project, the implementation focuses on deploying a highly available web application on Amazon Web Services (AWS). The deployment uses Amazon EC2, Apache HTTP Server, Amazon Machine Image (AMI), Launch Template, Classic Load Balancer, Auto Scaling Group, Amazon CloudWatch, and Amazon SNS.

The implementation follows Infrastructure as a Service (IaaS) principles, allowing cloud resources to be provisioned on demand while ensuring scalability, fault tolerance, and continuous availability.

6.2 AWS Management Console

The AWS Management Console is used to create and manage all cloud resources.

Steps

- Sign in to AWS Management Console.
- Select the Mumbai Region (ap-south-1).
- Navigate to the EC2 Dashboard.

6.3 Creating an EC2 Instance

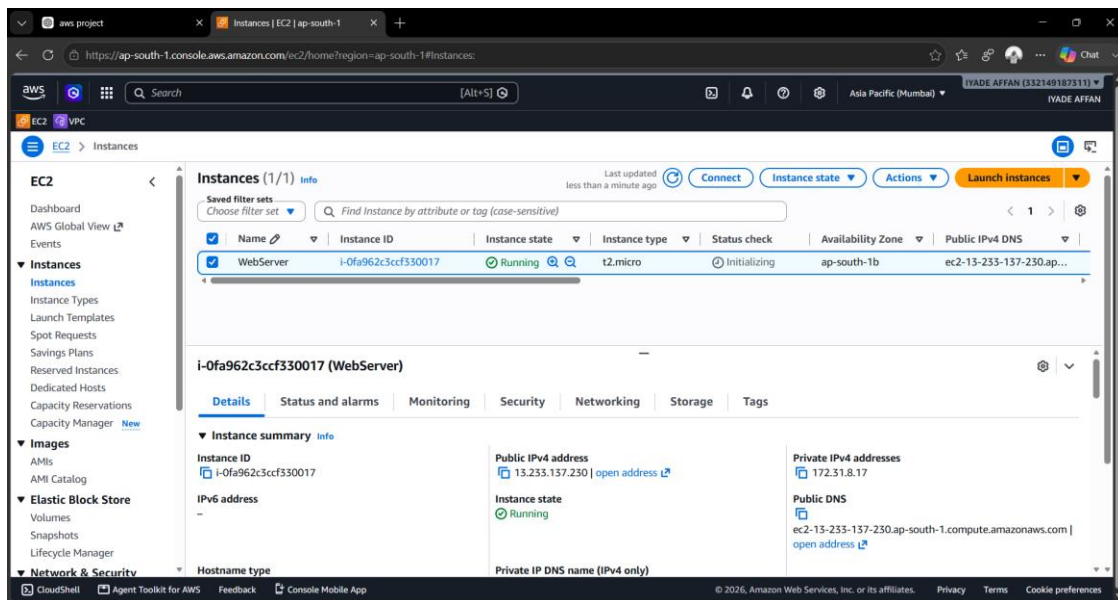
An EC2 instance is launched to host the web application.

Configuration

Parameter	Value
Instance Name	WebServer
Operating System	Amazon Linux 2
Instance Type	t2.micro (Free Tier)
Key Pair	web-key.pem
Storage	8 GB gp2
Security Group	WebSG
HTTP	Enabled
SSH	Enabled

Procedure

- Open EC2 Dashboard.
- Click Launch Instance.
- Enter the instance name as WebServer.
- Select Amazon Linux 2 AMI.
- Choose t2.micro.
- Create or select an existing Key Pair.
- Create a Security Group allowing ports 22, 80, and 443.
- Launch the instance.



Screenshot 6.1: EC2 Instance Running in the ap-south-1 (Mumbai) Region

6.4 Connecting to EC2

The EC2 instance is accessed securely using SSH.

Command

```
ssh -i web-key.pem ec2-user@<Public-IP>
```

After successful authentication, the Linux terminal appears.

6.5 Installing Apache Web Server

Apache is installed to host the website.

Commands

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
```

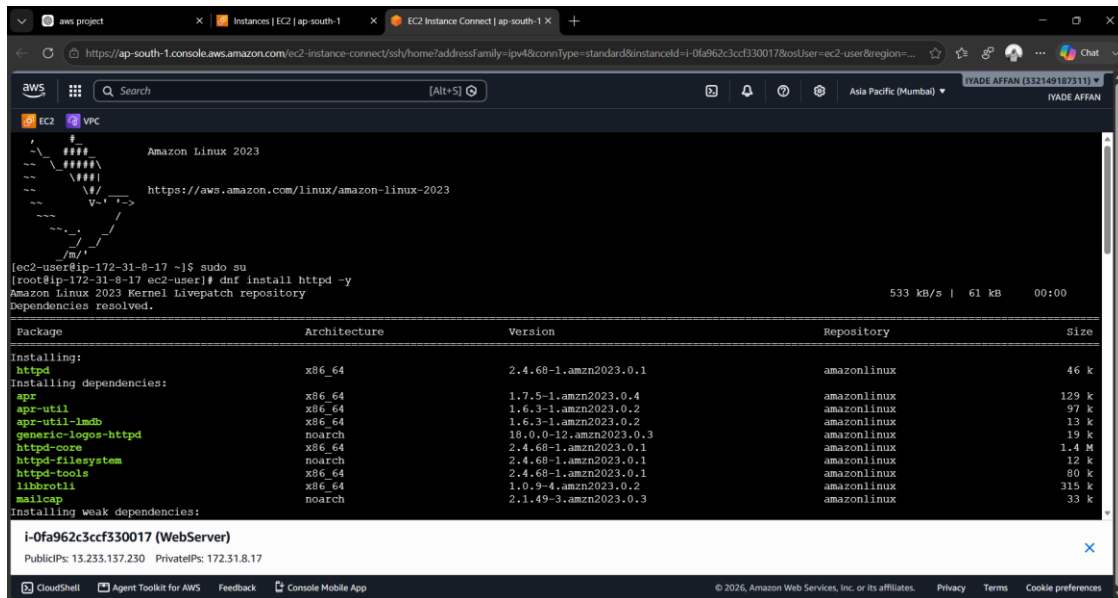


```
sudo systemctl enable httpd
```

Verification

```
systemctl status httpd
```

If the service is active, Apache has been installed successfully.



Screenshot 6.2: EC2 Instance Connect Session Showing Apache Installation on Amazon Linux 2023

6.6 Deploying the Website

The Bootstrap website is copied into Apache's web root directory.

Command

```
sudo cp -r * /var/www/html/
or
sudo unzip website.zip
sudo cp -r website/* /var/www/html/
```

Permission

```
sudo chmod -R 755 /var/www/html
```

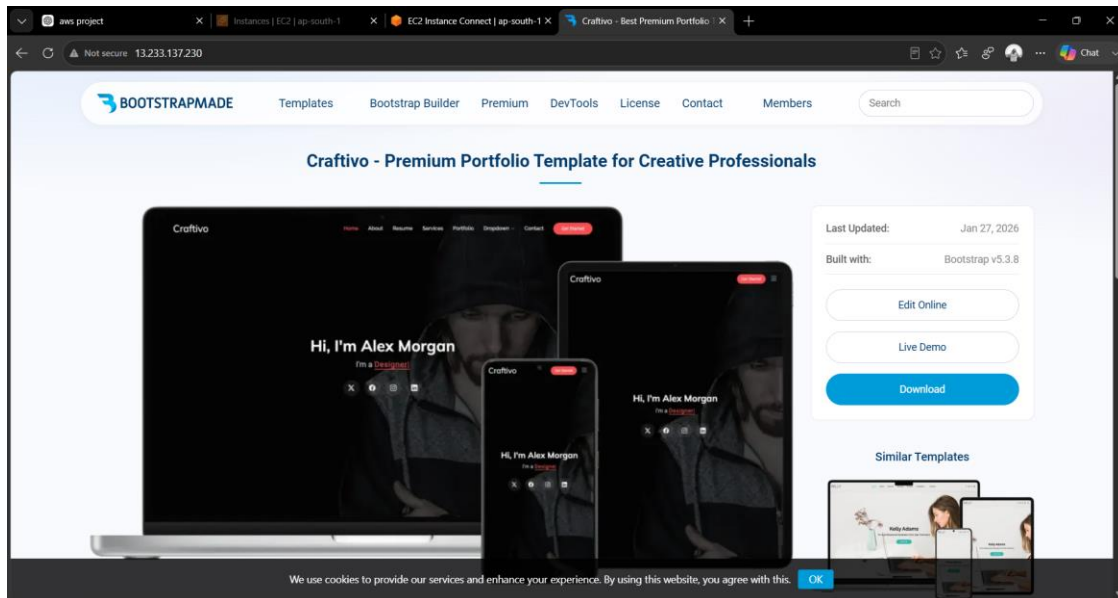
Restart Apache

```
sudo systemctl restart httpd
```

Open Browser

```
http://<Public-IP>
```

The website should load successfully.



Screenshot 6.3: Bootstrap-Based Website Successfully Hosted and Accessed via the EC2 Public IP

6.7 Creating an Amazon Machine Image (AMI)

Once the server is fully configured, an AMI is created.

Steps

- Select EC2 Instance.
- Click Actions.
- Select Image and Templates.
- Choose Create Image.
- Enter the image name: WebServer-AMI.
- Click Create Image.

The AMI preserves:

- Operating System
- Apache Installation
- Website Files
- Security Configuration

6.8 Creating Launch Template

Launch Templates automate EC2 creation.

Configuration

Parameter	Value
Template Name	WebTemplate
AMI	WebServer-AMI
Instance Type	t2.micro
Security Group	WebSG
Key Pair	web-key.pem

Steps

- Open EC2 Dashboard.
- Select Launch Templates.
- Click Create Launch Template.
- Choose the created AMI.
- Configure remaining settings.
- Save the template.

6.9 Creating Classic Load Balancer

The Classic Load Balancer distributes traffic across EC2 instances.

Configuration

Parameter	Value
Name	WebCLB
Protocol	HTTP
Port	80
Health Check	HTTP:80

Steps

- Open EC2 Dashboard.
- Load Balancers.
- Create Load Balancer.
- Choose Classic Load Balancer.
- Configure Listener.
- Configure Health Check.
- Attach Security Group.
- Create Load Balancer.

6.10 Creating Auto Scaling Group

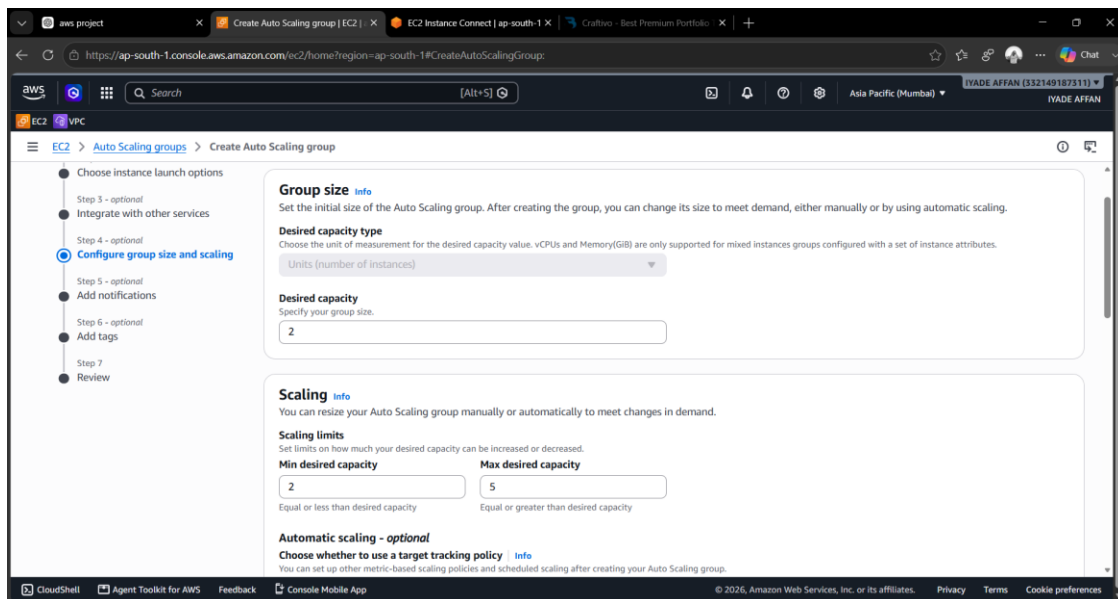
The Auto Scaling Group maintains multiple running instances.

Configuration

Parameter	Value
Name	Web-ASG
Launch Template	WebTemplate
Minimum Capacity	2
Desired Capacity	2
Maximum Capacity	5

Steps

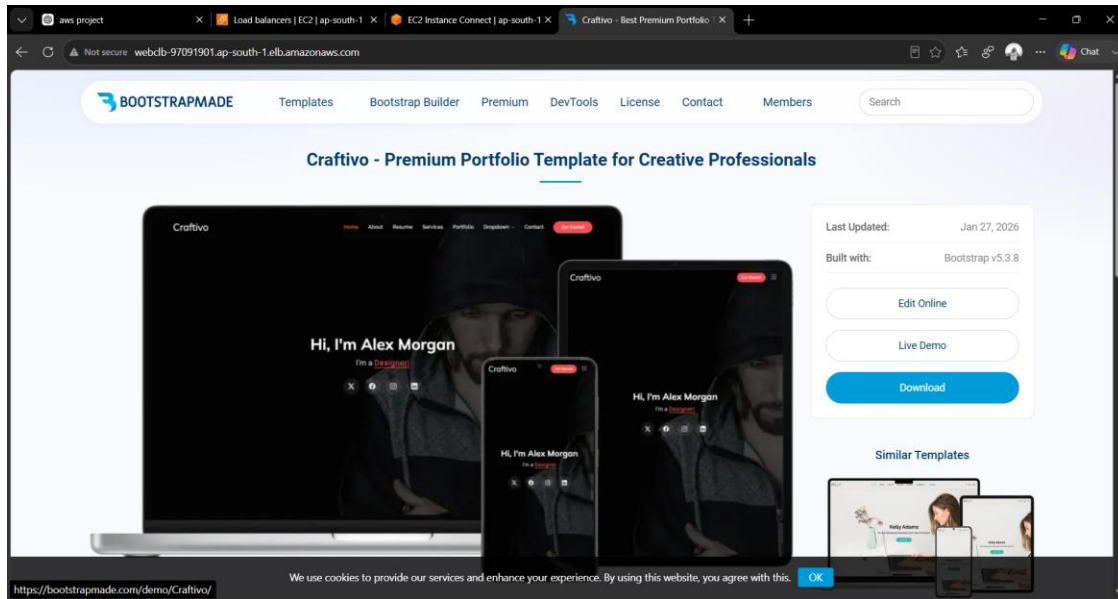
- Open Auto Scaling Groups.
- Click Create.
- Select Launch Template.
- Configure Capacity.
- Attach Load Balancer.
- Create Auto Scaling Group.



Screenshot 6.4: Auto Scaling Group – Group Size and Scaling Configuration

6.11 Verifying Website through the Load Balancer

Once the Auto Scaling Group and Classic Load Balancer are configured, the website is verified by accessing the Load Balancer DNS name in place of a single instance's public IP. This confirms that traffic is routed through the load balancer to the healthy EC2 instances registered behind it.



Screenshot 6.5: Website Successfully Accessed through the Classic Load Balancer DNS Name (webclb-97091901.ap-south-1.elb.amazonaws.com)

6.12 Configuring CloudWatch Alarm

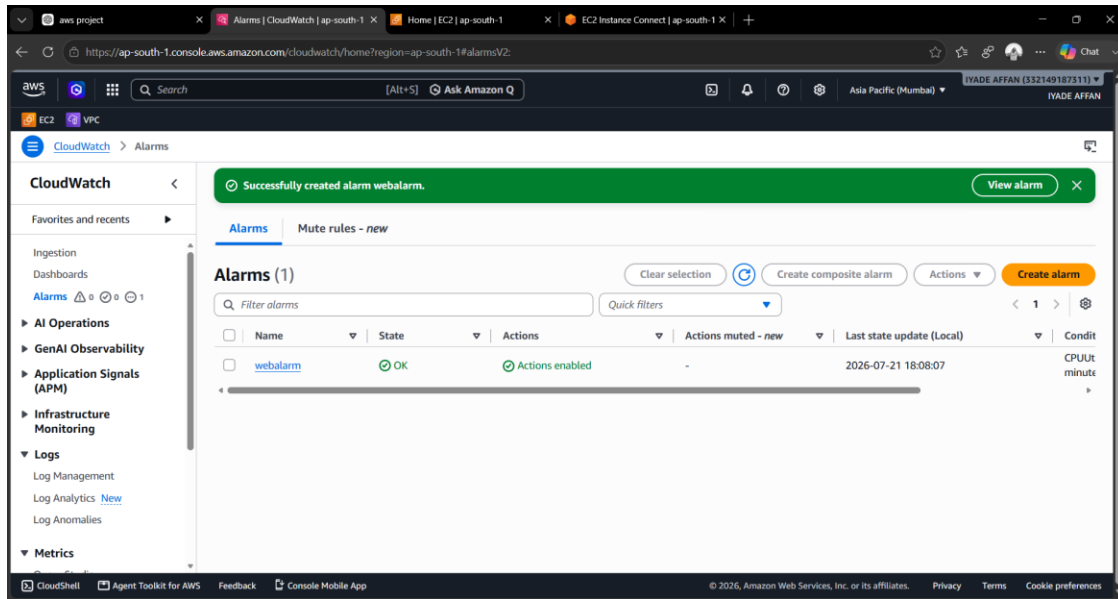
CloudWatch continuously monitors CPU utilization.

Alarm Configuration

Parameter	Value
Alarm Name	webalarm
Metric	CPU Utilization
Threshold	70%
Period	5 Minutes

Steps

- Open CloudWatch.
- Create Alarm.
- Select EC2 Metric.
- Choose CPU Utilization.
- Set Threshold.
- Configure SNS Notification.



Screenshot 6.6: CloudWatch Alarm "webalarm" Successfully Created

6.13 Creating SNS Topic

SNS sends email notifications.

Configuration

Parameter	Value
Topic Name	WebAlert
Protocol	Email
Endpoint	Administrator's Email Address

Steps

- Open SNS Console.
- Create Topic.
- Add Email Subscription.
- Confirm Email.

6.14 High Availability Testing

The application's fault tolerance is verified by terminating one running EC2 instance.

Test Procedure

- Launch two EC2 instances using Auto Scaling.
- Access the application through the Load Balancer.
- Terminate one instance manually.

- Observe Auto Scaling.
- Verify a new instance is launched automatically.
- Access the application again.

Expected Result

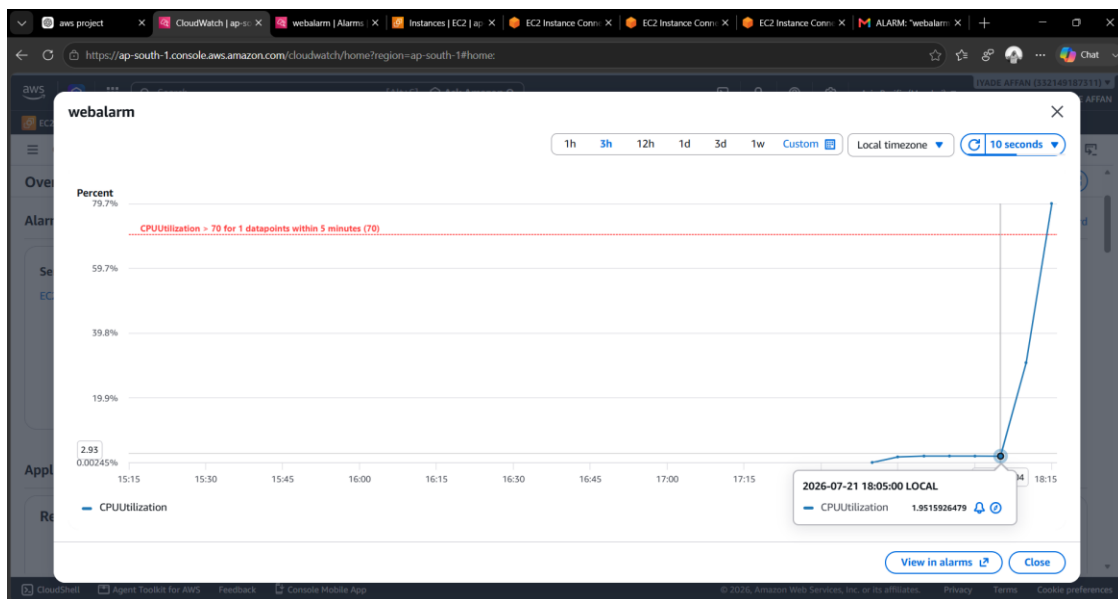
- Auto Scaling detects the unhealthy instance.
- A replacement instance is launched automatically.
- The Load Balancer routes traffic only to healthy instances.
- Users continue accessing the application without interruption.

6.15 Monitoring Results

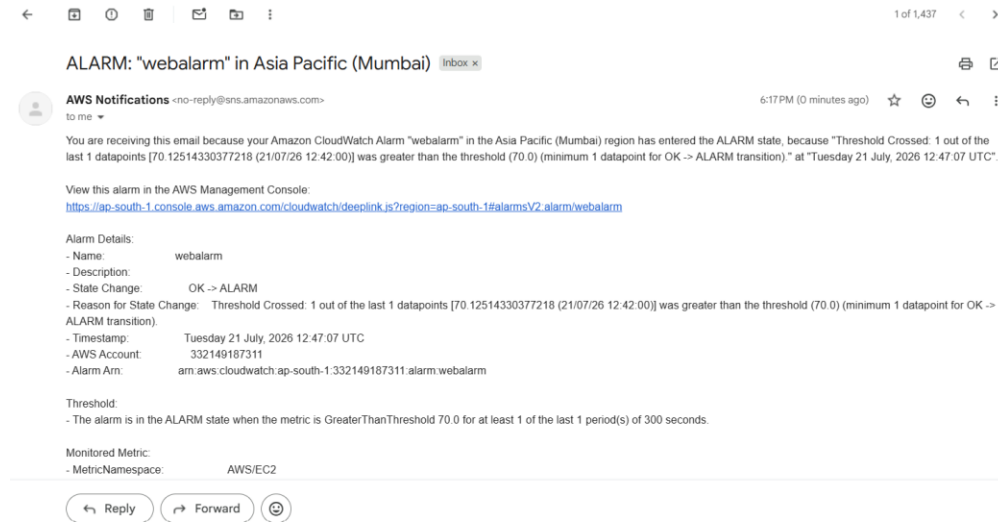
CloudWatch graphs display:

- CPU Utilization
- Network Traffic
- EC2 Status
- Alarm History

Whenever CPU usage exceeds the threshold, CloudWatch triggers the alarm and SNS sends an email notification.



Screenshot 6.7: CloudWatch Monitoring Graph Showing CPU Utilization Crossing the 70% Threshold



Screenshot 6.8: SNS Alarm Notification Email Received for "webalarm"

6.16 Implementation Summary

The implementation successfully demonstrates the deployment of a highly available web application using AWS. Amazon EC2 hosts the application, Apache serves web content, AMI and Launch Templates simplify deployment, Classic Load Balancer distributes traffic, Auto Scaling ensures fault tolerance, CloudWatch monitors resource utilization, and SNS provides real-time alerts. The system automatically recovers from instance failures while maintaining uninterrupted application availability, fulfilling the project's objectives.

CHAPTER 7

TESTING AND RESULTS

7.1 Introduction

Testing is an essential phase in software development that verifies whether the developed system satisfies the specified functional and non-functional requirements. In cloud computing, testing ensures that the deployed infrastructure behaves correctly under different operating conditions, including normal operation, increased traffic, server failures, and monitoring events.

In this project, several tests were conducted to validate the performance of the AWS infrastructure. The deployed web application was tested for accessibility, load balancing, automatic scaling, fault tolerance, monitoring, and notification capabilities. The results confirm that the proposed system successfully maintains application availability while automatically recovering from failures.

7.2 Testing Objectives

The objectives of testing are:

- Verify successful deployment of the web application.
- Validate Apache Web Server functionality.
- Test Classic Load Balancer traffic distribution.
- Verify Auto Scaling automatic recovery.
- Monitor CPU utilization using CloudWatch.
- Validate SNS email notifications.
- Ensure uninterrupted application availability.
- Verify security configurations.

7.3 Testing Environment

Component	Configuration
Cloud Platform	Amazon Web Services (AWS)
Operating System	Amazon Linux 2
Web Server	Apache HTTP Server
Browser	Google Chrome
Load Balancer	Classic Load Balancer
Monitoring Tool	Amazon CloudWatch
Notification Service	Amazon SNS
Auto Scaling	Enabled

7.4 Functional Testing

Functional testing verifies that each AWS service performs its intended function.

Test Case 1 – EC2 Instance Launch

Field	Details
Test Case ID	TC-01
Objective	Launch EC2 Instance
Input	Launch Instance
Expected Result	EC2 Instance Running
Actual Result	Successfully Running
Status	Pass

Test Case 2 – Apache Installation

Field	Details
Test Case ID	TC-02
Objective	Install Apache
Expected Result	Apache Service Active
Actual Result	Apache Running Successfully
Status	Pass

Test Case 3 – Website Hosting

Field	Details
Test Case ID	TC-03
Objective	Host Website
Expected Result	Website Accessible
Actual Result	Website Loaded Successfully
Status	Pass

Test Case 4 – AMI Creation

Field	Details
Test Case ID	TC-04
Objective	Create AMI
Expected Result	AMI Available
Actual Result	Successfully Created
Status	Pass

Test Case 5 – Launch Template

Field	Details
Test Case ID	TC-05
Objective	Create Launch Template
Expected Result	Template Ready
Actual Result	Successfully Created
Status	Pass

Test Case 6 – Classic Load Balancer

Field	Details
Test Case ID	TC-06
Objective	Configure Load Balancer
Expected Result	Traffic Distributed
Actual Result	Successfully Balanced
Status	Pass

Test Case 7 – Auto Scaling Group

Field	Details
Test Case ID	TC-07
Objective	Create Auto Scaling Group
Expected Result	Multiple Instances Running
Actual Result	Successfully Running
Status	Pass

Test Case 8 – CloudWatch Alarm

Field	Details
Test Case ID	TC-08
Objective	Configure Alarm
Expected Result	Alarm Triggered
Actual Result	Alarm Generated
Status	Pass

Test Case 9 – SNS Notification

Field	Details
Test Case ID	TC-09
Objective	Receive Email Alert
Expected Result	Email Delivered
Actual Result	Successfully Received
Status	Pass

7.5 High Availability Testing

The primary objective of this project is to ensure uninterrupted application availability even if a server fails.

Test Procedure

- Launch two EC2 instances using Auto Scaling.
- Attach both instances to the Classic Load Balancer.
- Access the website through the Load Balancer DNS.
- Manually terminate one EC2 instance.
- Observe the Auto Scaling Group.
- Verify that a replacement instance is launched.
- Confirm that the website remains accessible.

Expected Result

- Load Balancer stops routing traffic to the failed instance.
- Auto Scaling launches a replacement instance.
- Application remains available without manual intervention.

Actual Result

The Auto Scaling Group successfully detected the failed instance and launched a new EC2 instance automatically. The Classic Load Balancer routed traffic only to healthy instances, ensuring uninterrupted access to the web application.

Status: Pass

7.6 CloudWatch Monitoring Test

CloudWatch continuously monitored CPU utilization during application execution.

Monitoring Metrics

- CPU Utilization
- Network Traffic

- Disk Activity
- Instance Health

CloudWatch successfully recorded the performance metrics and displayed them through graphical dashboards, as shown in Screenshot 6.7.

7.7 SNS Notification Test

The SNS topic was subscribed using an email address.

When the CloudWatch alarm exceeded the configured CPU threshold:

- CloudWatch generated an alarm.
- SNS immediately sent an email notification.
- The administrator received the alert successfully, as shown in Screenshot 6.8.

7.8 Load Balancer Verification

The Classic Load Balancer distributed incoming HTTP requests among healthy EC2 instances.

Observation

- Requests were evenly distributed.
- Failed instances were removed automatically.
- Response time remained stable.
- Website accessibility was maintained.

7.9 Performance Analysis

Parameter	Observation
Website Availability	100% during testing
EC2 Recovery	Automatic
Load Distribution	Successful
CloudWatch Monitoring	Continuous
SNS Notification	Successful
Auto Scaling	Successful
Manual Intervention	Not Required

7.10 Result Analysis

The implementation successfully met all project objectives. The deployed infrastructure remained highly available throughout testing. Auto Scaling automatically replaced failed instances without affecting users. The Classic Load Balancer effectively distributed traffic among healthy instances, preventing overload on any single server.

CloudWatch continuously monitored resource utilization, while SNS provided immediate notifications when alarm conditions occurred. These results demonstrate the effectiveness of AWS services in creating scalable, fault-tolerant, and reliable cloud infrastructure suitable for enterprise applications.

7.11 Summary

The testing phase confirms that the proposed AWS architecture functions as expected under various operating conditions. All functional tests, load balancing tests, monitoring tests, and high-availability tests passed successfully. The deployed web application remained accessible during server failures, validating the effectiveness of Auto Scaling, Classic Load Balancer, CloudWatch, and SNS in providing continuous service availability.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

Cloud computing has revolutionized the way organizations deploy, manage, and maintain applications by providing scalable, reliable, and cost-effective infrastructure. Modern web applications require continuous availability, automatic recovery, and efficient resource utilization to meet growing user demands. Traditional hosting environments often fail to provide these capabilities due to their dependence on single-server architectures and manual administration.

The objective of this project was to design and implement a Highly Available Web Application Deployment on Amazon Web Services (AWS) using cloud-native services such as Amazon EC2, Amazon Machine Image (AMI), Launch Template, Classic Load Balancer (CLB), Auto Scaling Group (ASG), Amazon CloudWatch, and Amazon Simple Notification Service (SNS). The implementation successfully demonstrated how these services work together to create a scalable, fault-tolerant, and reliable cloud infrastructure.

The web application was initially deployed on an Amazon EC2 instance running Amazon Linux 2 with Apache HTTP Server. An Amazon Machine Image was created after configuring the application environment, allowing identical instances to be launched whenever required. A Launch Template simplified instance creation by storing reusable configuration details such as the AMI, instance type, security group, and key pair.

The Classic Load Balancer effectively distributed incoming HTTP requests among multiple EC2 instances, preventing server overload and improving application responsiveness. The Auto Scaling Group continuously monitored instance health and automatically replaced unhealthy instances without requiring manual intervention. This ensured uninterrupted service availability even during simulated server failures.

Amazon CloudWatch provided continuous monitoring of system metrics such as CPU utilization, while Amazon SNS delivered real-time email notifications whenever predefined thresholds were exceeded. These monitoring capabilities enabled administrators to detect and respond to infrastructure events quickly.

During testing, the application successfully remained available after terminating one of the running EC2 instances. The Auto Scaling Group immediately launched a replacement instance, and the Classic Load Balancer redirected traffic only to healthy servers. CloudWatch alarms and SNS notifications were also triggered as expected, validating the monitoring and alerting mechanisms.

Overall, the project successfully achieved all the stated objectives by demonstrating:

- Deployment of a web application on AWS.
- Successful hosting using Apache HTTP Server.
- Reusable server creation through Amazon Machine Images.
- Automated deployment using Launch Templates.
- Load distribution using Classic Load Balancer.
- Automatic recovery using Auto Scaling Group.
- Continuous infrastructure monitoring using CloudWatch.
- Email notifications through Amazon SNS.
- High availability with minimal downtime.
- Scalable cloud infrastructure suitable for real-world applications.

The project provides practical experience with AWS cloud services and demonstrates industry-standard practices for building highly available and fault-tolerant web applications.

8.2 Future Enhancement

Although the proposed system successfully provides a highly available web application deployment, several enhancements can be implemented to improve performance, scalability, security, and automation.

1. Application Load Balancer (ALB)

The existing Classic Load Balancer can be replaced with an Application Load Balancer to support advanced routing features, path-based routing, host-based routing, and improved performance.

2. HTTPS Implementation

SSL/TLS certificates issued through AWS Certificate Manager (ACM) can be configured to encrypt communication between users and the web application.

Benefits include:

- Secure communication
- Data encryption
- Improved user trust
- Better search engine ranking

3. Amazon Route 53 Integration

Route 53 can be used for domain registration and DNS management.

Advantages include:

- Custom domain names
- Health checks
- DNS failover

- Global traffic routing

4. Amazon RDS Integration

Currently, the project hosts only a static web application. Future implementations can integrate Amazon Relational Database Service (RDS) to support dynamic applications.

Possible databases include:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- SQL Server

5. Docker Containerization

The application can be containerized using Docker, making deployment more portable and consistent across environments.

Advantages include:

- Faster deployment
- Simplified updates
- Improved portability
- Better resource utilization

6. Kubernetes (Amazon EKS)

Amazon Elastic Kubernetes Service (EKS) can orchestrate containerized applications, providing:

- Automatic scaling
- Self-healing
- Rolling updates
- High availability

7. CI/CD Pipeline

A Continuous Integration and Continuous Deployment (CI/CD) pipeline can automate application deployment.

Tools include:

- GitHub
- Jenkins
- GitHub Actions
- AWS CodePipeline

- AWS CodeDeploy

Benefits:

- Faster deployment
- Reduced manual errors
- Continuous delivery
- Version control

8. Infrastructure as Code (IaC)

Instead of manually creating AWS resources, Infrastructure as Code tools such as Terraform or AWS CloudFormation can automate infrastructure provisioning.

Advantages:

- Repeatable deployments
- Version-controlled infrastructure
- Reduced configuration errors
- Faster provisioning

9. Advanced Monitoring

Additional AWS monitoring services can be integrated:

- AWS X-Ray
- AWS CloudTrail
- AWS Config
- Amazon GuardDuty

These services improve observability, auditing, and security monitoring.

10. Cost Optimization

Future versions of the project can reduce operational costs using:

- Spot Instances
- Savings Plans
- Reserved Instances
- AWS Cost Explorer
- Trusted Advisor

8.3 Overall Project Outcome

The project demonstrates that cloud-native technologies significantly improve application availability, scalability, and reliability compared to traditional single-server hosting.

The integration of Amazon EC2, Auto Scaling Group, Classic Load Balancer, CloudWatch, and SNS provides an effective solution for deploying enterprise-grade web applications capable of handling failures with minimal downtime.

This implementation serves as a strong foundation for future cloud-native applications and provides practical knowledge of AWS infrastructure management, monitoring, and automation.

REFERENCES

1. Amazon Web Services Documentation. Amazon EC2 User Guide.
<https://docs.aws.amazon.com/ec2/>
2. Amazon Web Services Documentation. Auto Scaling User Guide.
<https://docs.aws.amazon.com/autoscaling/>
3. Amazon Web Services Documentation. Elastic Load Balancing User Guide.
<https://docs.aws.amazon.com/elasticloadbalancing/>
4. Amazon Web Services Documentation. Amazon CloudWatch User Guide.
<https://docs.aws.amazon.com/cloudwatch/>
5. Amazon Web Services Documentation. Amazon SNS Developer Guide.
<https://docs.aws.amazon.com/sns/>
6. Amazon Web Services Documentation. IAM User Guide.
<https://docs.aws.amazon.com/iam/>
7. Apache Software Foundation. Apache HTTP Server Documentation.
<https://httpd.apache.org/>
8. Rajkumar Buyya, Christian Vecchiola, and S. Thamarai Selvi. Mastering Cloud Computing. McGraw-Hill Education.
9. Thomas Erl. Cloud Computing: Concepts, Technology & Architecture. Pearson Education.
10. Ian Foster, Yong Zhao, Ioan Raicu, and Shiyong Lu. Cloud Computing and Grid Computing 360-Degree Compared. IEEE.

APPENDIX

Appendix A – AWS Commands Used

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
sudo cp -r * /var/www/html/
sudo chmod -R 755 /var/www/html/
sudo systemctl restart httpd
```

Appendix B – AWS Resources Created

Resource	Name
EC2 Instance	WebServer
AMI	WebServer-AMI
Launch Template	WebTemplate
Auto Scaling Group	Web-ASG
Classic Load Balancer	WebCLB
CloudWatch Alarm	webalarm
SNS Topic	WebAlert

Appendix C – Screenshots Included

- AWS Console – EC2 Instance Running
- EC2 Instance Connect – Apache Installation
- Hosted Website via EC2 Public IP
- Auto Scaling Group – Group Size and Scaling Configuration
- Website Accessed via Classic Load Balancer DNS
- CloudWatch Alarm Created (webalarm)
- CloudWatch Monitoring Graph
- SNS Alarm Notification Email